XML in PHP 5

ApacheCon

November 17, 2003. Sin City, Baby!

Adam Trachtenberg <adam@trachtenberg.com>

Who Am I?

Who am I?

I'm Adam Trachtenberg.

- Programming in PHP since php/fi
- Co-author PHP Cookbook (O'Reilly 2003)
- Author PHP 5 Essentials (O'Reilly 2004)

I've been programming in PHP for quite a few years -- started out back when it was still php/fi (aka PHP 2). I recently wrote PHP Cookbook for O'Reilly & Associates and now I'm working on a new book for O'Reilly called PHP 5 Essentials.

My web site is at http://www.trachtenberg.com

But enough of this this, let's move on to the code.

Intro

What's New XMLwise in PHP5?

- Completely Rewritten
- libxml2 & libxsl Based
- Standards Compliant
- SimpleXML

The current state of XML support in PHP isn't so hot. You can do most of what you need -- SAX, DOM, XSLT, even some XPath, but nothing's standardized. Each extension uses a different library: expat, libxml2, Sablotron, libxsl. So getting up and running is quite a challenge.

On top of that, the extensions don't play well together, you can't take a DOM object and use it with the XSLT extension -- you need to use the special DOM XSLT functions instead.

In PHP 5, this is all fixed. We've standardized on just using libxml2 for XML and libxsl for XSL. All the extensions have been completely rewritten to not only work in unison, but also to take advantage of PHP'5 superior object model and language capabilities, like error handling.

Also the old DOM extension didn't always follow the W3 function naming conventions, so you couldn't pull DOM calls from, say, a Java project and quickly port them to PHP. That's been fixed.

Finally, there's a totally new XML extension called SimpleXML that's totally awesome. It's a -- simple -- way of reading and editing XML documents that lets you do 95 percent of what's possible with only 5 percent of the headache.

Topics

What's Will You Learn Today?

- SimpleXML
- -- Amazon REST
- -- Before SAX
- -- After SimpleXML
- -- XPath Query
- DOM
- -- RSS 0.9 and 1.0
- -- Create
- -- Read
- -- Read w/ Namespaces
- -- XPath Query
- -- XSLT Transform

I'm going to spend most of the next hour on two main topics: SimpleXML and DOM.

To demonstrate the ease-of-use of SimpleXML, I'm going to do a before-and-after presentation showing how to query Amazon.Com using REST and then parse the resulting data. First, I'm going to show how it's done using PHP 4 (or 5) and its SAX interface. Then, I'm going to show you how to do it using SimpleXML.

I think this will let everyone quickly see why SimpleXML rocks. :)

After that, I'll show how XPath support is built into SimpleXML

For the second half, I'll move onto DOM. My examples here are creating and reading RSS both 0.9 (aka Simple RSS) and 1.0 (aka Complex RSS, with namespaces and RDF and Dublin Core, and all that jazz). Once that's done, we'll quickly run by DOM and XPath and also DOM and XSLT

SimpleXML

- Easiest way to use XML.
- foreach your way through the document.
- Better than 80/20. It's 95/5.
- I <heart> SimpleXML.

Like I said before, SimpleXML is by far the easiest way to read basic XML documents I've ever seen. It's totally intuitive and what I use in any project that I can.

There's no need to set up callback functions, like with SAX. Instead, SimpleXML, like DOM, parses the XML document into a tree. Then, instead of using all those bulky accessor functions, like you must do with DOM, you can just foreach your way through the document. I'll show how to do this in a few slides.

SimpleXML embraces KISS: Keep It Simple, Stupid. You can't do everything with SimpleXML, like read in processing instructions or create comments, but you can do all the commonly occuring XML tasks and without all the headache caused when you need to support the entire XML specification.

SimpleXML in Action

Amazon.Com & REST

- Query Amazon.Com for book information using REST.
- Is everyone familiar with REST?
- What about Amazon.Com?
- Somewhat real life and somewhat complex.

My SimpleXML example is querying Amazon for books information using REST. REST stands for REpresentational State Transfter and it's one way of doing Web services. (There's also SOAP and XML-RPC.) When you use REST, you reuse existing tools, like HTTP and XML processing extensions instead of using a toolkit that pushes yet another layer on top of existing protocols, like SOAP and XML-RPC.

This example is "good" because it's actually something you'd do in "real life." This makes it somewhat complex, since you can't wave your hands at all the hard details, but I'm still able to not make it too difficult, so I can finish before my time is up.

Before: REST & SAX

In the old days, people used SAX. Here is a PHP 4 program to query Amazon using REST by invoking cURL and then parsing the data with PHP's OO interface to SAX:

As you can see, it's long. And complex. And not really fun to write for such an easy task. But, that's what you got to do:

I'm not going to even bother to explain it because either you know it, or you don't. If you know it, it's not interesting. If you don't, it'll take me the whole hour to explain it, and it's not that interesting.

```
#!/usr/local/php
<?php
$base ='http://xml.amazon.com/onca/xml3';
$query_string = '';

$params = array( 'ManufacturerSearch' => "O'Reilly",
                 'mode'               => 'books',
                 'sort'               => '+salesrank',
                 'page'               => 1,
                 'type'               => 'lite',
                 'f'                  => 'xml',
                 't'                  => 'trachtenberg-20',
                 'dev-t'              => 'XXXXX',
               );

foreach ($params as $key => $value) {
   $query_string .= "$key=" . urlencode($value) . "&";
}

$url = "$base?$query_string";

$c = curl_init($url);
curl_setopt($c, CURLOPT_RETURNTRANSFER, 1);
$hits = curl_exec($c);
curl_close($c);

$obj = new Amazon_parser;
$xml = xml_parser_create();
xml_set_object($xml, $obj);
xml_set_element_handler($xml, 'start_element', 'end_element');
xml_set_character_data_handler($xml, 'character_data');
xml_parser_set_option($xml, XML_OPTION_CASE_FOLDING, false);
xml_parse($xml, $hits);
xml_parser_free($xml);

class Amazon_parser {
```

```
  var $tag;
  var $item;

  function start_element($parser, $tag, $attributes) {
    // Each new item lives inside a Details container
    if ('Details' == $tag) {
      // Make a new Item
      $this->item = new Amazon_item;

      // Loop through the attributes
      foreach ($attributes as $name => $value) {
        if (isset($this->item->$name)) {
          $this->item->$name .= trim($value);
        }
      }
    } elseif (!empty($this->item)) {
      $this->tag = $tag;
    }
  }

  function end_element($parser, $tag) {
    // Each product ends with a closing Details tag
    if ('Details' == $tag) {
      $this->item->display();
      unset($this->item);
    }
  }

  function character_data($parser, $data) {
    if (!empty($this->item) and $data = trim($data)) {
      // only add pre-declared character data elements
      if (isset($this->item->{$this->tag})) {
        if(is_array($this->item->{$this->tag})) {
          $this->item->{$this->tag}[] = $data;
        } else {
          $this->item->{$this->tag}.= $data;
        }
      }
    }
  }
}

class Amazon_item {

  // attributes and character data fields we want
  var $ProductName = '' ;
  var $Author = array();
  var $OurPrice = '';
  var $url = '';
  var $ImageUrlSmall = '';
```

```
  function display() {
    printf('<div style="clear:left; width: 300px; padding:5px;
            margin:5px; background:#ddd;">
            <a href="s"><image src="s"
            alt="%s" align="left"/></a>
            <b>%s</b><br/>
            %s<br/>
            Amazon.com Price: %s<br/>
            </div>',

            $this->url, $this->ImageUrlSmall,
            htmlspecialchars($this->ProductName),
            htmlspecialchars($this->ProductName),
            htmlspecialchars(join(' and ', $this->Author)),
            htmlspecialchars($this->OurPrice)
          );
  }
}
?>
```

Before: REST & SAX: Results

Here's a picture of the results, after they've been turned into HTML and and viewed in a browser.

Ain't it pretty?

Now lets tackle the same problem using SimpleXML:



```
<?php
$base ='http://xml.amazon.com/onca/xml3';
$query_string = '';

$params = array( 'ManufacturerSearch' => "O'Reilly",
                 'mode'               => 'books',
                 'sort'               => '+salesrank',
                 'page'               => 1,
                 'type'               => 'lite',
                 'f'                  => 'xml',
                 't'                  => 'trachtenberg-20' ,
                 'dev-t'              => 'XXXXX' ,
               );

foreach ($params as $key => $value) {
   $query_string .= "$key=" . urlencode($value) . "&";
}

$url = "$base?$query_string";

$hits = simplexml_load_file($url);

foreach($hits->Details as $hit) {
    $a = $hit->Authors->Author;
    $a = (is_array($a)) ? join(' and ', $a) : $a;

    printf('<div style="clear:left; width: 300px; padding:5px; margin:5px; background:#ddd;">
            <a href="s"><image src="s" alt="s" align="left"/></a><b>s</b><br/>
            %s<br/>
            Amazon.com Price: %s<br/></div>',

            $hit->url, $hit->ImageUrlSmall, $hit->ProductName, $hit->ProductName,
            join(' and ', $a),
            $hit->OurPrice
          );
?>
```

As you can tell, it's much shorter now.

Let's work through it bit by bit:

The first half has nothing at all to do with XML and is just us creating a query string that we can pass to Amazon. With REST you send HTTP requests to URIs and receive XML in return.

The $params array is a bit associative array that we turn in to a HTTP GET request with the foreach loop by building up a query string and URLEncoding the values.

Then, instead of using cURL, we use SimpleXML's simplexml_load_file() function and pass the URL. Since SimpleXML supports PHP's streams functionality, it recognizes the parameter as a URL and makes a HTTP request for us. In the process it also creates a new SimpleXML object loaded up with the XML from Amazon.com.

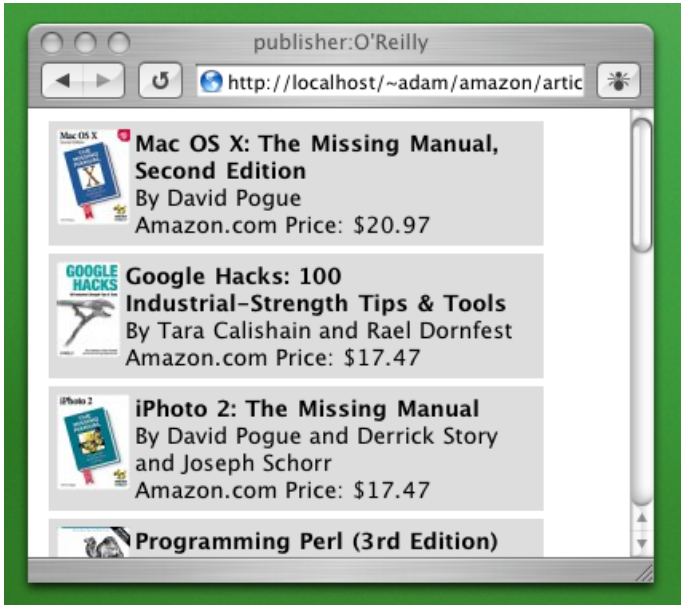
Now, all we need to do it iterate through the object using another foreach loop. Every book Amazon returned in this REST XML document is wrapped within a Details tag. There are 10 total, so we just use $hit->Details as $hit and this makes $hit an object whose properties are the attributes and child elements of each Details element.

The sprintf() function does all the same formatting we did before and lets us generate the same results:

REST and SimpleXML

After: REST & SimpleXML: Results

Ta da! Everything's the same, but with a lot less hassle.



XPath

SimpleXML also supports XPath

XPath is a W3 method of querying XML docs and receiving XML snippets in return. It's the same language used in XSLT to create template matching logic.

In this case, we load up a simple RSS document and make some queries:

```
<?php
$s = simplexml_load_file('rss0.9.xml');

// find and print all <title> elements
foreach($s->xsearch('//title') as $t) { print "$t\n"; }

// find and print every other <title> element
// living below an <item subject="PHP"> element
foreach($s->xsearch('descendant::item[attribute::subject="PHP"]/
          title[position() mod 2 = 1]') as $t) {
   print "$t\n";
}
?>
```

To make an XPath query, call xsearch() on an XML object and pass the XPath query as a parameter.

The first example finds all title tags and then prints them out. The second example finds ever other title tag that also has a parent item tag with a subject attribute set to PHP.

Needless to say, you can make lots of complex requests using XPath.

DOM

DOM is the mother of all XML extensions

- W3 Standard.
- Platform- and Language-Neutral.
- Completely comprehensive.
- And a pain in the ass.

DOM -- Document Object Model -- is a W3 standard for working with HTML and XML documents. If you've used DHTML, then you've used DOM. If you've used XML, then you've probably used DOM.

DOM is the elephant of XML processing extensions: it lets you do EVERYTHING. This is why it's great. It's also why it sucks. Super complex, but can be worth it, if you need to push XML to its limits. Also used for writing XML (assuming you don't want to just use print() functions) because SAX and SimpleXML don't support element creation.

DOM has the benefits of being completely platform- and language-neutral. So, in theory, the same DOM logic you used in you Java application can also be used in your Perl script, and then in your PHP program. PHP 4's DOM support wasn't quite up-to-spec, but PHP 5's follows along.

(There are actually 3 DOM levels, PHP supports what libxml2 supports, which is a lot, but not all 3 levels completely.)

DOM Details

DOM lets you create / edit / append / delete:

Like I said, you can do lots of things with DOM.

- Documents: One of those XML things
- Document Fragments: Almost XML
- Elements: <tag>
- Attributes: %attribute="value"
- Text: Stuff inside elements
- Comments: <!-- Comment -->
- Processing Instructions: <php... ?>
- CDATA: <![CDATA[...]]>
- Entities: &entity;

Creating XML w/DOM

Creating XML elements using DOM

DOM, like I said before, lets you create XML. This example adds a new RSS item to an existing feed stored in rss.xml:

Adding a new RSS item to an existing feed

```
<?php
$dom = domDocument::load('rss.xml');
$item = $dom->documentElement->appendChild(new domElement('item'));

$title = $item->appendChild(new domElement('title', 'PHP.net'));
$link = $item->appendChild(new domElement('link', 'http://www.php.net/'));
$description = $item->appendChild(new domElement('description', 'PHP Home Page'));

$dom->save('rss.xml');
?>
```

First you load the document using domDocument::load(). This places it into a tree, like SimpleXML does. Then you create new elements using domElement(). Passing one argument like, 'item', makes a wrapper tag. Passing two args, like 'title' and 'PHP.net' makes a tag called title with text in the middle of 'PHP.net'. (Text is also CDATA or character data.)

After you create an element, you still need to append it to the doucument in the location where you want it to live. That's why we pass the objects returned from domElement directly to appendChild, so they're not just created, but added to the tree.

Finally, you save the document by calling the save() method on your DOM object. This converts the internal tree back into an XML document and places it on disk.

Reading XML w/DOM

You can also read XML using DOM. As you can see, it's not as easy as with SimpleXML:

```
<?php
$dom = domDocument::load('rss0.9.xml');
$items = $dom->getElementsByTagname('item');

foreach ($items as $item) {
        $title = array_pop($item->getElementsByTagname('title'));
        $title_text_value = $title->firstChild->nodeValue;

        $link = array_pop($item->getElementsByTagname('link'));
        $link_text_value = $link->firstChild->nodeValue;

        $description = array_pop($item->getElementsByTagname('description'));
        $description_text_value = $description->firstChild->nodeValue;

        print "<p><a href=\"$link_text_value\">$title_text_value</a><br />\n";
        print "$description_text_value</p>\n";
}
?>
```

To find all tags, call getElementsByTagname() and pass the tagname as your parameter. This returns an array of elements which you can iterate through:

Inside the foreach(), we again call getElementByTagname to get our three tags: title, link, and description. To get the last element (which is also the first element, there's only one per item), we array_pop() it off of the array. Then, to access the text inside the element, use access the firstChild->nodeValue method.

Like I said, DOM is wordy.

Finally, we print out links at the bottom of the loop.

Reading Namespaced XML w/DOM

All of the examples so far have not had any XML namespaces. But, since XML does have namespaces, DOM supports them:

This is the same example as before, except now we're looking at an RSS 1.0 document, so it's an RDF document with about 5 different namespaces. The data we want is in the default rss namespace, located at http://purl.org/rss/1.0/.

```
<?php
$dom = domDocument::load('rss1.0.xml');

$rss = 'http://purl.org/rss/1.0/';
$items = $dom->getElementsByTagNameNS($rss, 'item');

foreach ($items as $item) {
        $title = array_pop($item->getElementsByTagnameNS($rss, 'title'));
        $title_text_value = $title->firstChild->nodeValue;

        $link = array_pop($item->getElementsByTagnameNS($rss, 'link'));
        $link_text_value = $link->firstChild->nodeValue;

        $description = array_pop($item->getElementsByTagnameNS($rss, 'description'));
        $description_text_value = $description->firstChild->nodeValue;

        print "<p><a href=\"$link_text_value\">$title_text_value</a><br />\n";
        print "$description_text_value</p>\n";
}
?>
```

Everything is the same as last time, but now we use getElementsByTagnameNS() instead. (Note the NS at the end of the function name.) This method takes 2 args: the namespace and the tag. Since the NS is a nasty long URL, I've assigned it to the $rss variable up above. Then, to find 'item' in the RSS namespace, you call getElementsByTagNameNS($rss, 'item').

Similar changes are made for title, link, and description and that's all there is to it.

DOM and XPath

If you don't like getElementsByTagname() and getElementsByTagnameNS(), you can use XPath instead:

This XPath query also searches against an RSS 1.0 document, so you can see how XPath and DOM handle namespaces:

```
<?php
$dom = domDocument::load('rss1.0.xml');

$xpath = new domXPath($dom);
$xpath->register_namespace('rss', 'http://purl.org/rss/1.0/');
$rss = 'http://purl.org/rss/1.0/';

$items = $xpath->query('//rss:item');

foreach ($items as $item) {
        $title = array_pop($item->getElementsByTagnameNS($rss, 'title'));
        $title_text_value = $title->firstChild->nodeValue;

        $link = array_pop($item->getElementsByTagnameNS($rss, 'link'));
        $link_text_value = $link->firstChild->nodeValue;

        $description = array_pop($item->getElementsByTagnameNS($rss, 'description'));
        $description_text_value = $description->firstChild->nodeValue;

        print "<p><a href=\"$link_text_value\">$title_text_value</a><br />\n";
        print "$description_text_value</p>\n";
}
?>
```

There's a new object introduced here: domXPath(). It's used to make XPath queries in conjunction with DOM. The constructor takes a domDocument and returns an object you can query.

Next, we register the namespace using the register_namespace() method on the XPath object. This lets use associate the string 'rss' with the longer URI. Since XPath embeds both the namespace and the element name into one string, we can't do use the two parameter syntax from getElementsByTagnameNS().

Finally, there's the XPath query itself: "//rss:item". It finds all item tags located inside the rss namespace. The semi-colon (:), separates the namespace from the element -- it acts as a delimiter.

Like getElementsByTagname(), query() returns an array of matching elements. Inside the foreach(), we still need to use old style functions, but that may not be true forever. Again, these extensions are in beta, so this limitation may be overcome by the presentation.

DOM and XSLT

DOM also works with XSLT

The new DOM extension has integrated XSLT functionality. Just like with XPath, we use a new object. This one is called "xsltprocessor"

This example harkens back to the earlier Amazon REST examples, but now instead of using SAX or SimpleXML, we're using DOM XSLT.

```
<?php
// Query Amazon
$xml = domDocument::load($url);

// Load local XSL stylesheet
$xsl = domDocument::load('amazon.xsl');

// Make XSLT object; load in stylesheet
$xslt = new xsltprocessor();
$xslt->importStylesheet($xsl);

// Transform!
print $xslt->transformToXML($xml);
?>
```

The hard part is writing the XSLT stylesheet, which is left for a different session. The easy part is using DOM XSLT.

First, you load in the XML and XSLT (which is also XML, of course) into DOM objects with domDocument::load. It doesn't matter that $url is the old HTTP GET request from earlier and amazon.xsl lives on our hard drive, they both work fine thanks to PHP and streams.

Next, we make a new XSLT object and load in the stylesheet by calling importStylesheet() and passing in its DOM object representation.

Now everything is primed and ready. To perform the transformation and print the results, call transformToXML() on the XSLT object and pass in the XML to be transformed. This triggers the change.

Resources

For More Information

Thanks for reading this talk. Here are a few links for the W3 specs. There's actually very little documention on PHP 5 and XML, so I can't point you anywhere for that.

- DOM Spec: http://www.w3.org/DOM/
- XPath: http://www.w3.org/TR/xpath
- XSL(T?): http://www.w3.org/Style/XSL/

SHAMELESS PLUG: This will be all documented (in nicer language and more detail) in PHP 5 Essentials, which should be out in early 2004. Check out O'Reilly's web site for more details in few months. The book will also cover all the other new PHP 5 topics, not just XML.

Index

Who Am I? 2
Intro 3
Topics 4
SimpleXML 5
SimpleXML in Action 6
REST and SAX 7
REST and SAX 10
REST and SimpleXML 11
REST and SimpleXML 13
XPath 14
DOM 15
DOM Details 16
Creating XML w/DOM 17
Reading XML w/DOM 18
Reading Namespaced XML w/DOM 19
DOM and XPath 20
DOM and XSLT 21
Resources 22

This presenation has ended.
Click anywhere to return.

_

